Konzeption und Evaluation eines Multi-Agenten-Systems zur automatisierten Migration von imperativer LO-VC Logik zu deklarativer AVC Syntax am Beispiel der msg systems ag

Torben Dörrer

22. April 2026

Inhaltsverzeichnis

1 Einleitung **1**

2 Related Work **2**

3 Experteninterview **6**

3.1 Zielsetzung und Erkenntnisinteresse 6
3.2 Methodisches Forschungsdesign 6
3.3 Datenerhebung und Auswertung 8
3.4 Ergebnisse der Befragung 9
3.4.1 Kognitive Altlasten und historischer Code 10
3.4.2 Paradigmenwechsel und syntaktisches Umdenken . . . 10
3.4.3 Fragmentierung manueller Prozessschritte 10
3.4.4 Limitierungen bestehender Standard-Tools 11
3.4.5 Manueller Aufwand und Fehleranfälligkeit im Testprozess 11
3.4.6 Zeitliche Metriken und Identifikation von Flaschenhälsen 11
3.5 Implikationen für die Systemarchitektur 12
3.5.1 Funktionale Anforderungen aus der Expertenperspektive 12
3.5.2 Architektonische Paradigmen und Beratungsansatz . . 12
3.5.3 Konzeptionelle Notwendigkeit einer Multi-Agenten-Struktur 13
3.5.4 Quantitative Zielvorgaben für die Systemvalidierung . . 13

Anhang Anhang **16**

Anhang Datenauswertung der Experteninterviews **17**

Kapitel 1

Einleitung

Beispieltext für die Einleitung.

Kapitel 2

Related Work

Die Automatisierung komplexer Software-Migrationen erfordert ein tiefgreifendes Verständnis sowohl der proprietären Zieldomäne als auch der neuesten informationstechnischen Möglichkeiten. Da die wissenschaftliche Forschungslage an der direkten Schnittstelle zwischen proprietärer SAP-Migration und generativer Künstlicher Intelligenz bislang äußerst dünn besetzt ist, ordnet dieses Kapitel die vorliegende Arbeit anhand hochaktueller Publikationen aus angrenzenden Disziplinen in den Stand der Technik ein. Hierzu werden zunächst die spezifischen prozessualen Hürden bei der Transformation von SAP-Konfigurationsmodellen beleuchtet, um die Limitierungen aktueller Standardwerkzeuge aufzuzeigen. Daran anknüpfend wird der Einsatz von Large Language Models (LLMs) für die Code-Migration diskutiert, wobei insbesondere die empirisch belegten Grenzen monolithischer Ansätze bei strikten Architekturvorgaben herausgearbeitet werden. Abschließend wird der aktuelle State of the Art zu Multi-Agenten-Systemen (MAS) vorgestellt, um die methodische Forschungslücke für die semantische Migration von proprietärer Constraint-Logik logisch herzuleiten.

Die Migration von etablierten ERP-Systemen auf SAP S/4HANA zwingt Unternehmen zur Überführung ihrer historisch gewachsenen Produktmodelle von der klassischen Variantenkonfiguration (LO-VC) in die Advanced Variant Configuration (AVC). Matilainen (2024) verdeutlicht in seiner Untersuchung zu S/4HANA-Produktdatenstrukturen, dass dieser Systemwechsel weit über ein rein syntaktisches Update hinausgeht. Aufgrund der exponentiell wachsenden Variantenvielfalt bei konfigurierbaren Materialien wird im AVC eine strikte Modellierung durch neue Konfigurationsprofile und harte deklarative Constraints zwingend erforderlich.

Ein kritischer Erfolgsfaktor der Migration ist dabei die vorgelagerte Datenbereinigung. Wie Matilainen (2024) aufzeigt, müssen historische Altlasten und ungenutzte Merkmale vor der Systemumstellung zwingend identifiziert

und entfernt werden, um die Performance und Struktur des neuen Systems nicht zu belasten. Obgleich SAP für diesen Prozess Standardwerkzeuge, wie beispielsweise Transition Workspaces und das Migration Cockpit, zur Verfügung stellt, stoßen diese bei der logischen Transformation an ihre Grenzen. Diese Werkzeuge arbeiten primär deterministisch und fokussieren sich auf die reine Massendatenübernahme. Die komplexe, semantische Restrukturierung von historischem Beziehungswissen in die deklarative AVC-Logik sowie die inhaltliche Bereinigung des Legacy-Codes verbleiben als stark fragmentierter, manueller Flaschenhals bei den Fachexperten. Die Literatur verdeutlicht somit, dass die konzeptionelle Vorbereitung und Logik-Transformation im aktuellen Industrie-Standard stark manuell geprägt ist, woraus sich ein zwingender Bedarf an intelligenten Automatisierungsansätzen ableitet.

Die rasanten Fortschritte im Bereich der Large Language Models (LLMs) haben die Möglichkeiten der automatisierten Code-Generierung und -Transformation grundlegend erweitert (Karabiyik, 2025). Dennoch belegen aktuelle Forschungsarbeiten, dass der direkte Einsatz von LLMs für die Migration komplexer Altsysteme – oft als „Single-Prompt-Ansatz“ oder „Monolithic Prompting“ bezeichnet – in realen Produktionsumgebungen kaum praktikable Ergebnisse liefert.

Die Ursache hierfür liegt primär in der Unfähigkeit monolithischer Ansätze, spezifische Architekturvorgaben und interne Framework-Abhängigkeiten zu berücksichtigen. Moti et al. (2026) demonstrieren in ihrer Untersuchung zum Framework LegacyTranslate, dass naive LLM-Übersetzungen bei der Migration industrieller Codebasen eine Kompilierrate von 0 % erreichen können. Während der generierte Code zwar syntaktisch korrekt erscheint, scheitert er an der fehlenden Integration notwendiger interner Schnittstellen (APIs) und domänenspezifischer Logik (Moti et al., 2026). Ähnliche Defizite dokumentieren Xu et al. (2025) im Kontext von MANTRA: Während spezialisierte Systeme hohe Erfolgsquoten erzielen, erreichen monolithische Basismodelle bei komplexen strukturellen Änderungen (wie dem „Move Method“-Refactoring) lediglich eine Erfolgsquote von 8,7 %.

Darüber hinaus identifizieren Karabiyik (2025) und Oueslati et al. (2026) konzeptionelle Schwächen in der Kontrollierbarkeit solcher „One-Shot“-Ansätze. Ohne eine Aufteilung in modulare Arbeitsschritte bleibt der Transformationsprozess eine intransparente „Blackbox“, die keine funktionalen Garantien bietet und bei längeren Interaktionen zu Halluzinationen neigt (Karabiyik, 2025; Oueslati et al., 2026). Die Literatur macht somit deutlich, dass für eine erfolgreiche semantische Transformation – insbesondere unter Berücksichtigung von restriktiven Vorgaben wie der SAP „Clean Core“-Strategie – eine Loslösung von statischen Einzelinstruktionen hin zu dynamischen, mehrstufigen Prozessen zwingend erforderlich ist.

Um die Unzulänglichkeiten monolithischer Ansätze zu überwinden, rückt die aktuelle Forschung zunehmend Multi-Agenten-Systeme (MAS) in den Fokus, die auf dem Prinzip der verteilten Kognition und spezialisierter Rollenverteilung basieren. Das Kernkonzept dieser Architekturen besteht darin, den Transformationsprozess in logisch abgegrenzte Teilaufgaben zu zerlegen, die von kooperierenden Agenten autonom bearbeitet werden (Karabiyik, 2025; Oueslati et al., 2026).

Ein wegweisendes Paradigma stellt hierbei die sequentielle Orchestrierung dar, wie sie im Framework RefactorGPT realisiert wird. Durch die Aufteilung in spezialisierte Rollen – von der Analyse der Strategie über die strukturelle Transformation bis hin zur Fehlerbehebung durch einen dedizierten „Fixer-Agent“ – wird der Migrationsprozess von einer intransparenten Blackbox in einen kontrollierbaren und modularen Workflow überführt (Karabiyik, 2025). Die Überlegenheit dieser Rollenspezialisierung wird durch das Framework RefAgent bestätigt: Oueslati et al. (2026) belegen empirisch, dass ein System aus spezialisierten Agenten für Planung, Generierung und Validierung die Fehlerquote und die Neigung zu Halluzinationen bei komplexen Interaktionen signifikant reduziert.

Ein zentrales Merkmal dieser modernen Architekturen ist die Integration autonomer Feedback-Schleifen zur Sicherstellung der funktionalen Korrektheit. Das System MANTRA nutzt hierfür einen „Repair Agent“, der mittels „Verbal Reinforcement Learning“ Compiler-Fehler und Test-Logs analysiert, um den generierten Code iterativ zu korrigieren. Dieser Ansatz ermöglichte eine Steigerung der Erfolgsquote bei komplexen Transformationen von 8,7 % auf 82,8 % (Xu et al., 2025).

Speziell für die Migration in restriktiven Unternehmensumgebungen verdeutlicht das Projekt LegacyTranslate Moti et al. (2026) zudem die Notwendigkeit des sogenannten „API Grounding“. Da die bloße Übersetzung der Syntax in industriellen Frameworks oft zu nicht kompilierbarem Code führt, integriert LegacyTranslate spezialisierte Agenten, die den Zielcode aktiv an interne Architekturvorgaben und Bibliotheken anpassen (Moti et al., 2026). Zusammenfassend lässt der aktuelle Stand der Forschung erkennen, dass die Kombination aus rollenbasierter Spezialisierung, kontextueller Anreicherung und iterativen Validierungszyklen die einzige methodische Antwort auf die Komplexität industrieller Legacy-Migrationen darstellt.

Trotz der empirisch nachgewiesenen Überlegenheit von Multi-Agenten-Systemen bei der Automatisierung von Code-Übersetzungen offenbart die aktuelle Literatur eine signifikante Lücke hinsichtlich proprietärer ERP-Ökosysteme. Während etablierte Frameworks wie MANTRA Xu et al. (2025) oder LegacyTranslate Moti et al. (2026) primär auf objektorientierte oder klassisch prozedurale Standardsprachen (wie Java oder PL/SQL) fokussiert sind, fehlt

bislang die Anwendung kollaborativer KI-Architekturen auf die domänenspezifische Constraint-Logik der SAP.

Wie die Untersuchung von Matilainen (2024) verdeutlicht, erfordert die Migration von LO-VC zu AVC nicht nur eine syntaktische Konvertierung, sondern eine tiefgreifende semantische Restrukturierung. Hierzu zählen insbesondere die inhaltliche Auflösung historischer, kognitiver Altlasten und die Überführung prozeduraler Vorbedingungen in ein streng deklaratives, mathematisches Lösungsnetzwerk. Bisherige wissenschaftliche MAS-Ansätze bieten keine architektonische Antwort auf dieses fachliche Umdenken und die strikten „Clean Core"-Richtlinien, die bei der SAP-Migration als harte Systemgrenzen fungieren.

Genau an dieser methodischen Schnittstelle setzt die vorliegende Arbeit an. Sie schließt die identifizierte Forschungslücke, indem sie die Konzepte arbeitsteiliger Agenten-Architekturen auf die spezifischen Herausforderungen der SAP-Variantenkonfiguration überträgt. Durch die Konzeption und Evaluierung eines spezialisierten Multi-Agenten-Systems zur semantischen Migration von LO-VC nach AVC wird ein Lösungsansatz entwickelt, der den aktuellen manuellen Flaschenhals der Logik-Transformation systematisch adressiert und das Systemverhalten aktiv an die moderne Zielarchitektur anpasst.

Kapitel 3

Experteninterview

In diesem Kapitel wird die Durchführung und Auswertung der Experteninterviews beschrieben. Die gewonnenen Erkenntnisse dienen als qualitative Basis für die Anforderungen an die spätere Systemumgebung.

3.1 Zielsetzung und Erkenntnisinteresse

Die Relevanz sowie die Kritikalität der Migration von LOVC zu AVC für Unternehmen wurden bereits im vorangehenden Kapitel dargelegt. Für eine KI-gestützte Automatisierung dieses Prozesses mittels eines Multi-Agenten-Systems ist die Kenntnis der theoretischen Differenzen zwischen LOVC und AVC allein unzureichend. Spezifische, historisch gewachsene Problemstellungen in der Entwicklungspraxis lassen sich nicht rein analytisch aus Systemdokumentationen ableiten und weisen oft eine hohe Kontextabhängigkeit auf. Zudem mangelt es an quantitativen Daten bezüglich des zeitlichen Migrationsaufwands, was die Validierung der Relevanz und Notwendigkeit einer Automatisierung erschwert.

Folglich ist die Durchführung von Experteninterviews zur Beantwortung der ersten Forschungsfrage essenziell. Dieses methodische Vorgehen dient der Identifikation impliziter Problemstellungen im msg-Kontext. Daraus werden sowohl quantitative Metriken für die Definition einer menschlichen Baseline und die spätere Evaluation des Modells generiert als auch funktionale Anforderungen an einen KI-basierten Migrationsassistenten abgeleitet.

3.2 Methodisches Forschungsdesign

Als methodisches Forschungsdesign zur Durchführung der Experteninterviews wird der semi-strukturierte Ansatz gewählt. Aufgrund der hohen Kom-

plexität der Migration von LOVC zu AVC, welche unter anderem durch menschliches Verhalten und undokumentiertes Wissen geprägt ist, bedarf es zur fundierten Durchdringung der Problemstellung neben quantitativen Methoden zwingend qualitativer Forschungsansätze. Diese methodische Notwendigkeit wird durch die einschlägige Grundlagenliteratur gestützt. (Seaman, 1999)

Die Wahl des semi-strukturierten Formats erweist sich hierbei als zielführend. Rein unstrukturierte Interviews gewähren den Befragten oftmals ein Übermaß an Spielraum, sind ressourcenintensiv und bergen das Risiko, dass essenzielle Themenkomplexe unberücksichtigt bleiben. Demgegenüber limitiert ein streng strukturiertes Vorgehen den Raum für unerwartete qualitative Erkenntnisse. (Seaman, 1999)

Da die Zielsetzung der Datenerhebung sowohl die Akquise antizipierter als auch unerwarteter Informationen umfasst, stellt das semi-strukturierte Interview das optimale Instrumentarium dar. Das Erhebungsdesign setzt sich folglich aus einer Kombination offener und spezifischer Fragen zusammen. Diese Konzeption zielt darauf ab, einerseits vorhersehbare Daten zu generieren – wie etwa zeitliche Metriken zur Etablierung einer menschlichen Baseline –, und andererseits unerwartete Erkenntnisse zu gewinnen. Letztere sind beispielsweise für die praxisnahe Anforderungsdefinition an ein KI-basiertes Migrationssystem von hoher Relevanz.

Hinsichtlich der Vorbedingungen an die befragende Person sowie an die Durchführung der Interviews gelten spezifische Prämissen. Während in diversen sozialwissenschaftlichen Disziplinen ein geringes Vorwissen der interviewenden Person oft ausreichend ist, postulieren Hove & Anda (2005) für den Bereich der Softwareentwicklung die Notwendigkeit eines tiefgreifenden technischen Vorverständnisses. Zur Gewährleistung einer lückenlosen und korrekten Datenerfassung ist zudem die vollständige Audioaufzeichnung und anschließende Transkription der Interviews unerlässlich. (Hove & Anda, 2005)

Die Auswertung des Interviewtranskripts erfolgte durch einen systematischen Mixed-Methods-Ansatz, der Methoden der qualitativen und quantitativen Datenanalyse kombiniert. Um eine lückenlose Beweiskette ('Chain of Evidence') gemäß Runeson und Höst (2009) sicherzustellen, gliederte sich der Codierungsprozess in zwei Phasen: Zunächst wurde in Anlehnung an die strukturierende Inhaltsanalyse nach Mayring (2014) eine deduktive Kategorienanwendung durchgeführt. Hierbei wurden im Vorhinein definierte Codes, welche direkt aus den drei Forschungsfragen abgeleitet wurden, auf das Textmaterial angewendet, um dieses theoriegeleitet zu codieren.

Um gleichzeitig eine methodische Offenheit für unvorhergesehene praxisnahe Schmerzpunkte bei der Migration zu gewährleisten, wurde dieser

Schritt durch eine induktive Kategorienbildung (Mayring (2014)) ergänzt, bei der ergänzende Codes iterativ am empirischen Material entwickelt wurden. (siehe Anhang 3.5.4). Daran anschließend wurden aus den codierten Textstellen Field Memos generiert, welche die Kernaussagen der Befragung zusammenfassen und als Hypothesen für die in Kapitel 3.5. beschriebenen Anforderungen an die Systemarchitektur dienen. (Seaman, 1999)

Ziel dieser Auswertung war eine duale Erkenntnisgewinnung: Einerseits wurden durch gezieltes quantitatives Coding (Seaman (1999)) harte Metriken zum zeitlichen und manuellen Aufwand extrahiert, welche als menschliche Baseline für die spätere Modellevaluation des Multi-Agenten-Systems dienen. Andererseits wurden aus den identifizierten kognitiven und prozessualen Hürden funktionale Anforderungen an die zukünftige KI-Architektur abgeleitet. Diese Anforderungen stellen jedoch bewusst noch nicht die finale Systemarchitektur dar. Gemäß dem Prinzip der Methodentriangulation (Runeson und Höst (2009)) bilden die Erkenntnisse des Experteninterviews lediglich eine von drei Säulen, welche im weiteren Verlauf der Arbeit mit den Ergebnissen des praktischen Selbstversuchs (Kapitel X) und der theoretischen Grundlagenanalyse (Kapitel Y) synthetisiert werden, um die finale, valide Agenten-Architektur zu konstruieren.

Ein zentrales Qualitätsmerkmal qualitativer Fallstudien im Software Engineering stellt die Aufrechterhaltung einer lückenlosen Beweiskette (Chain of Evidence") dar. Im Kontext dieser Arbeit impliziert dies die transparente Ableitung der Notwendigkeit jedes einzelnen KI-Agenten aus den Rohdaten. Die methodische Nachvollziehbarkeit wird hierbei über den gesamten Prozess – von der Audioaufzeichnung über das codierte Transkript und die daraus resultierenden Field-Memos (Kernaussagen) bis hin zu den finalen funktionalen Anforderungen an die Systemarchitektur – gewährleistet. (Runeson & Höst, 2009)

3.3 Datenerhebung und Auswertung

Das Experteninterview wurde am 17.03.2026 mit einem erfahrenen Product Owner und Softwareentwickler der msg durchgeführt. Die Fachexpertise des Interviewpartners umfasst eine zwölfjährige Tätigkeit als Maschinenbauingenieur sowie eine mehr als sechsjährige Spezialisierung auf dem Gebiet der Variantenkonfiguration. Seit knapp vier Jahren liegt sein Tätigkeitsschwerpunkt auf der Betreuung von Kundenprojekten zur Variantenkonfiguration im msg-Kontext. Die Gesprächsdauer belief sich auf rund 45 Minuten. Die Befragung erfolgte durch den Autor dieser Arbeit mit solidem technischem Vorwissen in den Bereichen LOVC und AVC. Als Erhebungsinstrument diente ein vor-

ab definierter Interviewleitfaden. Dieser umfasste offene Fragen zur Akquise unerwarteter Erkenntnisse sowie spezifische Fragestellungen zum gezielten Informationsgewinn und der Erhebung von quantitativen Metriken. Letztere dienen der Beantwortung der ersten Forschungsfrage sowie der späteren Modellevaluation.

Mit Einverständnis des Experten erfolgte eine vollständige Aufzeichnung sowie automatische Transkription des Gesprächs über die integrierte Funktion von MS Teams. Zur Gewährleistung der Vertraulichkeit wurde das Transkript im Anschluss manuell anonymisiert. Zur Optimierung der Lesbarkeit fand daraufhin eine KI-gestützte Glättung des Textes unter strikter Wahrung der inhaltlichen Integrität statt. Eine finale manuelle Überprüfung des Resultats stellte sicher, dass durch die KI-Bearbeitung keine inhaltlichen Verzerrungen eingeführt wurden.

Die anschließende Datenauswertung erfolgte mittels des in Kapitel 3.2 beschriebenen Codierungsverfahrens. Die initiale Codeliste wurde deduktiv aus den drei Kernfragen des Interviewleitfadens abgeleitet. Zur Gewährleistung einer lückenlosen Beweiskette von der Datenerhebung bis zur Systemarchitektur erfolgte eine Gliederung der Codes in drei Hauptkategorien: 1. Erfassung der manuellen Prozessschritte, 2. Quantifizierung des zeitlichen Aufwands und Identifikation von Engpässen sowie 3. systematische Erfassung kognitiver Hürden und typischer menschlicher Fehlerquellen.

Darauf aufbauend wurde das Transkript in einem iterativen Prozess durch induktive Code-Generierung erweitert, um unerwartete Erkenntnisse strukturiert zu erfassen. Den Abschluss des Codierungsprozesses bildete die inhaltliche Gruppierung der Codes zur Extraktion der Kernaussagen. Aus diesen Ergebnissen wurden schließlich belastbare funktionale Anforderungen an das Multi-Agenten-System abgeleitet sowie die zeitlichen Metriken zur Definition der menschlichen Baseline generiert.

3.4 Ergebnisse der Befragung

Die nachfolgende Darstellung fasst die zentralen Erkenntnisse der qualitativen Inhaltsanalyse zusammen, welche durch die Synthese der Field Memos in thematische Schwerpunkte gegliedert wurden. Diese deskriptiven Ergebnisse bilden den empirischen Ist-Zustand des Migrationsprozesses ab und dienen als Grundlage für die spätere Anforderungsanalyse und die Definition der menschlichen Baseline.

3.4.1 Kognitive Altlasten und historischer Code

Die Auswertung der Interviewdaten verdeutlicht, dass historisch gewachsene LOVC-Modelle durch eine hohe Dichte an ungenutzten Altlasten charakterisiert sind. Merkmale, Prozeduren und Beziehungswissen verbleiben oft im System, ohne im aktuellen Modellierungskontext eine funktionale Relevanz zu besitzen. Zudem führen über Jahre stetig anwachsende Werteräume zu einer erhöhten Komplexität. Für den Migrationsprozess bedeutet dies, dass eine unreflektierte 1:1-Konvertierung in den Advanced Variant Configurator (AVC) aufgrund dieser behindernden Altlasten in der Praxis unzureichend ist. Für den Entwickler resultiert daraus eine erhebliche kognitive Belastung, da eine zeitintensive manuelle Analyse zur Trennung von aktiver Logik und „totem Code“ zwingend erforderlich ist, bevor die eigentliche syntaktische Transformation beginnen kann.

3.4.2 Paradigmenwechsel und syntaktisches Umdenken

Über die rein syntaktische Ebene hinaus zeigt die Befragung einen fundamentalen Paradigmenwechsel in der Modellierungstechnik auf. Dieser wird maßgeblich durch die „Clean Core“-Strategie der SAP getrieben, wodurch Eigenentwicklungen wie Z-Functions nicht mehr unterstützt werden. Da sich das Systemverhalten im AVC selbst bei identischer Syntax massiv ändern kann – etwa durch den Zwang zu klassifizierten Merkmalen oder die globale Wirkung von Constraints – führt eine bloße Übernahme alter Logiken häufig zu abweichenden Konfigurationsergebnissen oder Performance-Einbußen. Der Migrationsprozess erfordert daher ein aktives Umdenken: Entwickler müssen die ursprüngliche Intention des Legacy-Codes in Gänze verstehen und diese semantisch mit den neuen Bordmitteln des AVC abbilden.

3.4.3 Fragmentierung manueller Prozessschritte

Trotz der Existenz von Standard-Tools zur Massenanlage von Objekten bleibt der Migrationsprozess in weiten Teilen fragmentiert und durch manuelle Arbeitsschritte geprägt. Während Automatisierungen die Anlage von Basisdaten übernehmen können, verbleibt die komplexe intellektuelle Durchdringung der Logik beim Entwickler. Dieser muss Vorbedingungen manuell auswerten, in tabellenbasierte Strukturen überführen und abschließend auf Konsistenz prüfen. Dieser Workflow erzwingt häufig Medienbrüche und degradiert den Experten zu einer „manuellen Übersetzungsmaschine“, was die Fehleranfälligkeit erhöht und den Prozess verlangsamt.

3.4.4 Limitierungen bestehender Standard-Tools

Die Auswertung zeigt, dass bestehende Standardlösungen wie das „SAP Migration Cockpit“ primär die rein technischen Datenübernahmen (wie die Massenanlage von Objekten) abdecken. Bei der komplexen Logik-Transformation entstehen jedoch funktionale Lücken. Der Experte äußert den Bedarf nach einer prozessualen Unterstützung auf einer „beratenden Flugebene“, welche Zusammenhänge aufzeigt und Optimierungspotenziale identifiziert. Es mangelt im aktuellen Ist-Zustand an Werkzeugen zur automatisierten Kommentierung von historischem Code, zur tiefgehenden Analyse von Z-Functions sowie an proaktiven Ratschlägen zur Modellstrukturierung. Eine rein statische Code-Konvertierung erweist sich für den Gesamtprozess somit als unzureichend.

3.4.5 Manueller Aufwand und Fehleranfälligkeit im Testprozess

Ein signifikanter Flaschenhals wurde im Bereich der Qualitätssicherung identifiziert. Das Testen migrierter Modelle erfolgt gegenwärtig weitgehend manuell, da bestehende Testtools oft nicht in der Lage sind, logische Veränderungen zwischen Alt- und Neusystem korrekt zu interpretieren. Dies führt zu einem massiven Aufwand beim händischen Abgleich von Testaufträgen und birgt ein entsprechendes Potenzial für menschliche Flüchtigkeitsfehler in der abschließenden Validierungsphase.

3.4.6 Zeitliche Metriken und Identifikation von Flaschenhälsen

Zur Definition einer menschlichen Baseline wurden im Rahmen der Befragung konkrete zeitliche Metriken erhoben. Die Migration eines mittleren Modells beansprucht demnach etwa drei bis vier Personentage, wobei Extremfälle bei komplexen Strukturen bis zu mehrere Monate beanspruchen können. Als primärer zeitlicher Flaschenhals wurde die Transformation der Vorbedingungswelt identifiziert, die etwa zwei Drittel (66 %) der gesamten Arbeitszeit beansprucht. Weitere 40 % des Aufwands entfallen auf die vorbereitende Analyse und den Aufbau unterstützender Strukturen wie Excel-Tabellen. Diese quantifizierten Werte bilden die Vergleichsvariablen für die spätere Evaluierung.

Die hier dargelegten empirischen Befunde skizzieren die aktuellen prozessualen und kognitiven Hürden der Migration. Sie bilden die konzeptionelle Ausgangslage, um im folgenden Unterkapitel konkrete Implikationen und Anforderungen an die zu entwickelnde Systemarchitektur abzuleiten.

3.5 Implikationen für die Systemarchitektur

Basierend auf den empirischen Befunden der qualitativen Inhaltsanalyse lassen sich spezifische Anforderungen und konzeptionelle Leitplanken für die Gestaltung der Systemarchitektur ableiten. In diesem Kapitel werden ausschließlich jene Implikationen dargelegt, die direkt aus der Expertenbefragung hervorgehen.

3.5.1 Funktionale Anforderungen aus der Expertenperspektive

Aus den identifizierten prozessualen Hürden ergeben sich zwingende funktionale Notwendigkeiten für das Systemdesign:

- **Semantische Analyse und Code-Bereinigung:** Da historische Altlasten die Migration behindern, muss die Architektur eine Komponente zur Identifikation von ungenutzten Merkmalen und Prozeduren enthalten. Ziel ist eine Reduktion der kognitiven Last durch eine automatisierte Vorab-Bereinigung des Legacy-Codes.

- **Automatisierte Logik-Transformation:** Angesichts des zeitlichen Flaschenhalses bei der Erstellung von Tabellen und Constraints muss der Fokus auf der automatisierten Generierung dieser Strukturen liegen. Das System muss in der Lage sein, die logische Restrukturierung von Vorbedingungen zu übernehmen, um den manuellen Aufwand in diesem Bereich zu minimieren.

- **Konsistenzprüfung im Testprozess:** Um die Fehleranfälligkeit bei der Validierung zu senken, wird eine Funktion zum Mapping alter Testkonfigurationen auf die Zielmodelle benötigt, die logische Abweichungen zwischen den Systemwelten erkennt.

3.5.2 Architektonische Paradigmen und Beratungsansatz

Der identifizierte Paradigmenwechsel in der Modellierungstechnik bedingt spezifische architektonische Eigenschaften:

- **Interaktive Assistenz (Consulting-Ansatz):** Das System darf nicht als rein deterministischer Konverter agieren. Gefordert ist eine interaktive Ebene, die Design-Entscheidungen begründet und dem Nutzer beratend zur Seite steht, um den Kontext der Transformation transparent zu machen.

- **Einhaltung von Modellierungsrichtlinien (Clean Core):** Die Architektur muss aktuelle SAP-Vorgaben wie die Clean-Core-Strategie und Performance-Best-Practices als Wissensbasis integrieren. Transformationen müssen proaktiv auf diese Zielarchitektur hin optimiert werden, statt lediglich die Quellsyntax abzubilden.

3.5.3 Konzeptionelle Notwendigkeit einer Multi-Agenten-Struktur

Die Heterogenität der Anforderungen – von der Analyse komplexer Z-Functions bis hin zur Validierung von Test-Sets – legt eine Verteilung der Aufgaben auf spezialisierte Rollen nahe. Ein monolithischer Ansatz erscheint unzureichend, um die benötigte Tiefe in den Bereichen Analyse, Beratung, Synthese und Test gleichzeitig abzudecken. Daraus ergibt sich die Notwendigkeit einer Multi-Agenten-Architektur, in der spezialisierte Agenten (Analyst, Syntaktiker, Tester und Consultant) kollaborativ zusammenarbeiten.

3.5.4 Quantitative Zielvorgaben für die Systemvalidierung

Die erhobenen Zeitmetriken definieren die Erfolgskriterien für die spätere Evaluierung der Architektur. Das System muss gegen die menschliche Baseline von drei bis vier Tagen pro Modell validiert werden. Insbesondere muss nachgewiesen werden, dass der primäre Flaschenhals der Vorbedingungs-Transformation (66 % des Gesamtaufwands) sowie die vorbereitende Analysephase (40 % des Aufwands) durch die automatisierte Unterstützung signifikant reduziert werden können.

Die hier dargelegten interviewbasierten Implikationen bilden zusammen mit den Erkenntnissen aus dem Selbsexperiment und der theoretischen Analyse die vollständige Anforderungsbasis für den anschließenden Systementwurf in Kapitel 4.

Literatur

Hove, S. E., & Anda, B. (2005). Experiences from Conducting Semi-Structured Interviews in Empirical Software Engineering Research. *11th IEEE International Software Metrics Symposium (METRICS'05)*, 10–pp. Verfügbar 31. März 2026 unter https://ieeexplore.ieee.org/abstract/document/1509301/

Karabiyik, M. A. (2025). RefactorGPT: A ChatGPT-based Multi-Agent Framework for Automated Code Refactoring. *PeerJ Computer Science*, *11*, e3257. Verfügbar 21. April 2026 unter https://peerj.com/articles/cs-3257/

Matilainen, A. (2024). Change Requirements on Product Data Structures in S/4HANA Implementation. Verfügbar 21. April 2026 unter https://osuva.uwasa.fi/items/8bf7ae52-e756-4fca-9a25-c0510c58ad34

Mayring, P. (2014). Qualitative Content Analysis: Theoretical Foundation, Basic Procedures and Software Solution. Verfügbar 7. April 2026 unter https://www.ssoar.info/ssoar/bitstream/handle/document/39517/ssoar-2014-mayring-Qualitative_content_analysis_theoretical_foundation.pdf

Moti, Z., Soudani, H., & Kogel, J. van der. (2026, 14. März). *LegacyTranslate: LLM-based Multi-Agent Method for Legacy Code Translation.* arXiv: 2603.14054 [cs]. https://doi.org/10.48550/arXiv.2603.14054

Oueslati, K., Lamothe, M., & Khomh, F. (2026, 4. März). *RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring.* arXiv: 2511.03153 [cs]. https://doi.org/10.48550/arXiv.2511.03153

Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, *14*(2), 131–164. https://doi.org/10.1007/s10664-008-9102-8

Seaman, C. B. (1999). Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Transactions on software engineering*, *25*(4), 557–572. Verfügbar 31. März 2026 unter https://ieeexplore.ieee.org/abstract/document/799955/

Xu, Y., Lin, F., Yang, J., Tse-Hsun, Chen & Tsantalis, N. (2025, 27. März). *MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration.* arXiv: 2503.14340 [cs]. https://doi.org/10.48550/arXiv.2503.14340

Anhang

Im Rahmen dieser Arbeit wurden folgende ergänzende Unterlagen erstellt:

- **Anhang A:** Datenauswertung der Experteninterviews (Excel)
- **Anhang B:** Leitfaden für die Befragung

Datenauswertung der Experteninterviews

Category Title	Marked Text
03_01_Kognitiv_Altlasten	In der LOVC-Welt war das etwas anders: Da konnten Merkmale irgendwo enthalten sein, auch wenn sie im Modell gerade nicht verwendet wurden.
03_01_Kognitiv_Altlasten	Wenn man ein Modell direkt konvertieren will, hat man oft Altlasten, die das Modell behindern, weil beispielsweise die Prozedur oder das Beziehungswissen nicht ausgeführt werden.
03_01_Kognitiv_Altlasten	Die Werteräume dieser Merkmale wachsen über die Zeit stetig an.
03_02_Kognitiv_Syntax_Umdenken	Viele Kunden haben sich zum Beispiel eigene Funktionen (Z-Functions) programmiert, die im Beziehungswissen aufgerufen werden. Diese werden jetzt nicht mehr unterstützt.
03_02_Kognitiv_Syntax_Umdenken	Es gibt zwar BAdIs, mit denen wir einiges implementieren können, aber in der Clean-Core-Welt, in der alle Kunden jetzt unterwegs sein wollen, versucht man so etwas zu vermeiden.
03_02_Kognitiv_Syntax_Umdenken	Ein weiterer Punkt ist die von SAP empfohlene Modellierungstechnik für den AVC, die sich grundlegend von der im LOVC unterscheidet.
03_02_Kognitiv_Syntax_Umdenken	Diese sollen wir im AVC nun nicht mehr nutzen, sondern stattdessen mit Constraints arbeiten.
03_02_Kognitiv_Syntax_Umdenken	Man kann nichts direkt übernehmen. Auch wenn die Syntax eins zu eins gleich bleibt, muss ein neues Objekt generiert werden, da in den Eigenschaften des Objekts hinterlegt ist, ob es sich um klassisches Beziehungswissen oder um AVC handelt.
03_02_Kognitiv_Syntax_Umdenken	Der LOVC kommt mit nicht klassifizierten Merkmalen klar, der AVC hingegen nicht.
03_02_Kognitiv_Syntax_Umdenken	AVC an gewissen Stellen anders verhält als der LOVC.
03_02_Kognitiv_Syntax_Umdenken	Systemverhalten hat sich leicht geändert. Dieses veränderte Verhalten macht es oft notwendig, den Code anders zu schreiben oder um Dinge zu ergänzen, die vorher nicht da waren.
03_02_Kognitiv_Syntax_Umdenken	Das ist ein grundlegender Unterschied im Systemverhalten, der zu abweichenden Konfigurationsergebnissen führen kann und den man bei der Migration berücksichtigen muss.
03_02_Kognitiv_Syntax_Umdenken	Im AVC kann ich das nicht mehr spezifisch definieren. Im AVC werden immer alle Merkmale zueinander eingeschränkt.
03_02_Kognitiv_Syntax_Umdenken	Wenn ich also genau dasselbe Constraint wie vorher nun im AVC aufrufe, resultiert daraus möglicherweise ein ganz anderes Systemverhalten.
03_02_Kognitiv_Syntax_Umdenken	Das ist ein extremer Performance-Killer, den man laut SAP vermeiden soll.
03_03_Kognitiv_Fehler_Menschlich	Und am Ende war die Datei trotzdem fehlerhaft.
04_01_Tool_Konkrete_Anforderung	Man könnte sich vorstellen, diesen Ergebnisvergleich zu automatisieren oder von einer KI durchführen zu lassen.

04_01_Tool_Konkrete_Anforderung	Die KI könnte hierbei vielleicht noch bestimmte Hürden der Testautomatisierung überwinden.
04_01_Tool_Konkrete_Anforderung	Vielleicht könnte ein KI-Bot sich die alten Aufträge schnappen und aus den drei Einzelaufträgen automatisch ein gebündeltes Set bilden.
04_01_Tool_Konkrete_Anforderung	Man könnte beispielsweise einen Parser entwickeln, der die Syntax anhand fester Regeln übersetzt. Die KI sehe ich mittlerweile eher auf einer höheren Flugebene als beratendes Tool, das Zusammenhänge aufzeigt und Verbesserungspotenziale im Modell identifiziert.
04_01_Tool_Konkrete_Anforderung	Wenn ich ein Analysetool hätte, das so etwas erkennt, wäre das toll.
04_01_Tool_Konkrete_Anforderung	Die KI könnte so etwas erkennen und Ratschläge geben, wie: "Du machst die pauschale Bewertung am Anfang, setze sie doch ans Ende" oder "Du hast hier x Überschreibungen, die du vermeiden könntest, wenn du die Bedingungsteile präziser einschränkst, damit Sonderfall B nicht mehr im Sonderfall A enthalten ist."
04_01_Tool_Konkrete_Anforderung	Die KI könnte dann feststellen: "Die Prozedur an Position 10 wird im Kontext der anderen Prozeduren nie relevant sein, du kannst sie herausnehmen oder musst sie anders einsortieren."
04_01_Tool_Konkrete_Anforderung	dass die KI auch weiterhin zum Übersetzen genutzt wird.
04_01_Tool_Konkrete_Anforderung	fände ich es aber sehr gut, wenn du diese beratende Flugebene in dein Tool integrieren könntest.
04_01_Tool_Konkrete_Anforderung	wenn ich als Nutzer die KI zu meinem aktuellen Modell oder einer Funktion befragen könnte.
04_01_Tool_Konkrete_Anforderung	Die KI könnte beispielsweise den vorhandenen Code kommentieren.
04_01_Tool_Konkrete_Anforderung	Eine KI könnte hier automatisch Kommentare generieren, weil sie die Merkmale und die zugehörigen Beschreibungstexte miteinander verknüpft.
04_01_Tool_Konkrete_Anforderung	Es wäre cool, wenn eine KI in diese Z-Functions hineinschauen und analysieren könnte, was diese Funktion genau macht. Darauf aufbauend könnte sie vorschlagen, wie man diese Funktion im neuen System über ein BAdI lösen könnte.
04_01_Tool_Konkrete_Anforderung	Was extrem wertvoll wäre, ist, wenn die KI den Schritt von Vorbedingungen hin zu Constraints übernehmen könnte.
01_01_Prozess_Schritt_Manuell	Also musste der Mitarbeiter von Hand Bewertungen für die drei Modelle vornehmen, damit am Ende alles zusammenpasst
01_01_Prozess_Schritt_Manuell	Meistens schränke ich dort in Tabellenform die Werte zueinander ein und definiere gültige Kombinationen von Merkmalswerten.
01_01_Prozess_Schritt_Manuell	Man lädt sein LOVC-Modell dort hinein und kann es dann auf die Eignung für den AVC analysieren.
01_01_Prozess_Schritt_Manuell	Das ist quasi der erste Schritt: Man bekommt seine To-Dos aufgezeigt, also was im Beziehungswissen angepasst werden muss, und das Tool übernimmt die Massenanlage des Beziehungswissens.

01_01_Prozess_Schritt_Manuell	kann es anschließend aufräumen.
01_01_Prozess_Schritt_Manuell	Dafür muss ich die ganzen Vorbedingungen, die an den Werten hängen, analysieren und auswerten, was sie aussagen.
01_01_Prozess_Schritt_Manuell	Dort erstelle ich dann ein tabellenbasiertes Beziehungswissen für das jeweilige Produkt,
01_01_Prozess_Schritt_Manuell	erst einmal völlig losgelöst vom vorhandenen Konfigurationsmodell aus einer neutralen Sicht verstehen.
01_01_Prozess_Schritt_Manuell	fände ich es aber sehr gut, wenn du diese beratende Flugebene in dein Tool integrieren könntest.
01_01_Prozess_Schritt_Manuell	Dann habe ich geprüft, ob sich das deckt oder ob ich einen Denkfehler habe.
01_02_Prozess_Toolbruch	Meistens baue ich mir dafür eine große Excel-Tabelle auf.
02_01_Zeit_Dauer_Konkret	Sie haben ein einfaches bis mittleres Modell in drei bis vier Tagen migriert.
02_01_Zeit_Dauer_Konkret	Ich würde sagen, das macht etwa 20 % der Arbeit aus, der Großteil entfällt auf die Wandlung der Vorbedingungen in Constraints.
02_01_Zeit_Dauer_Konkret	20 % für die Merkmalswelt, 40 % für die Analyse und den Excel-Aufbau und der Rest entfällt auf die eigentliche Umsetzung.
02_01_Zeit_Dauer_Konkret	ein Kollege hat dort vier Monate lang in Vollzeit eine Excel-Tabelle aufgebaut, nur um die Vorbedingungen eines einzigen Modells zu analysieren.
02_02_Zeit_Flaschenhals	Diese Vorbedingungswelt durch constraint-basierte Modellierung abzulösen, ist eigentlich die Hauptarbeit bei dieser Migration.
02_02_Zeit_Flaschenhals	Ein Großteil der Arbeit besteht dabei darin, die vorab genannten Vorbedingungen durch Constraints abzulösen.
02_02_Zeit_Flaschenhals	Dieser Transfer also syntaktische Vorbedingungen in eine Tabellenform zu bringen und diese Tabellen dann in Constraints umzuwandeln macht ungefähr zwei Drittel der Arbeit aus.
02_02_Zeit_Flaschenhals	der Großteil entfällt auf die Wandlung der Vorbedingungen in Constraints.
02_02_Zeit_Flaschenhals	zu großen Teilen wird es dann zu einer reinen Umsetzungsaufgabe.
02_02_Zeit_Flaschenhals	20 % für die Merkmalswelt, 40 % für die Analyse und den Excel-Aufbau und der Rest entfällt auf die eigentliche Umsetzung.
Explizite_Tool_Nennung	Wenn man sich für eine direktere Migration entscheidet, gibt es von SAP mittlerweile ein Migration Cockpit, das beim Umstieg hilft.
Explizite_Tool_Nennung	im Trace
Testen	Es ist rein manuelles Testen.
Testen	durchaus Strategien für automatisches Testen.
Testen	letztes Jahr für den Kunden ein Konzept für ein mögliches Testtool geschrieben.
Testen	Der Prozess sieht so aus: Man nimmt sich einen vorhandenen Testauftrag als Basis und nutzt eine Migrationstabelle für die Merkmale. Ich weiß also, was aus dem alten Merkmal A in meinem neuen Modell geworden ist beispielsweise Merkmal B. Damit gehe ich in die Konfiguration und lege einen Kundenauftrag im neuen System mit den neuen Merkmalsbewertungen an. Den Auftrag speichere ich ab und vergleiche dann den alten Kundenauftrag mit dem neuen.

Testen	Wir haben also die Konstellation, dass aus drei alten Objekten ein neues Objekt geworden ist. Diesen Schritt konnte das geplante Testtool bisher nicht abbilden.
Testen	Das Tool kann zwar das Fahrzeug in ein Testobjekt migrieren, aber die Batterie müsste von Hand nachkonfiguriert werden.
Komplexität_Modellgröße	Bei kleinen Modellen, wie die Kollegen sie aktuell betreuen, gibt es teilweise nur etwa 20 bis 30 Merkmale. Dort ist eine direkte Migration natürlich deutlich leichter möglich.
Limitierung_KI_Tool	haben wir letztes Jahr einen Proof of Concept für das Migrate-Tool gemacht. Dabei sind wir bei der reinen Syntaxübersetzung bereits auf Hürden gestoßen und haben uns gefragt, ob eine KI für die reine Syntaxübersetzung wirklich das richtige Werkzeug ist. In der Zwischenzeit haben wir auch andere Ansätze gesehen. Man könnte beispielsweise einen Parser entwickeln, der die Syntax anhand fester Regeln übersetzt. Die KI sehe ich mittlerweile eher auf einer höheren Flugebene als beratendes Tool, das Zusammenhänge aufzeigt und Verbesserungspotenziale im Modell identifiziert.